助听器课程设计报告

一、 背景及功能介绍：

随着世界人口老龄化，全球约有 4 亿人患有听力损失相关疾病，中国就占 1 亿人左右。听力损失导致了人们交流障碍，由此引发能力低下、心理疾病和阿尔兹海莫症等问题。目前，佩戴助听器是解决听力损失最有效的方法。

我们智能助听器的设计不仅能够提升听力，更关注老年人群体的体验，并结合了现代智能技术，以提供更加便捷的连接和更丰富的功能。

我们助听器主要的功能有

音乐播放：共五首歌曲，分别为小星星，暖暖，浙大校歌，我和我的祖国，爱的罗曼史，通过板子五个大开关（P2,R2,M4,N4,R1，即 S4-0）控制播放所选歌曲，具体为小星星（00001），暖暖（00010），浙大校歌（00100），我和我的祖国（01000），爱的罗曼史（10000）。

主动报警：当老人跌倒或遇到其他危险时，推上控制音乐播放所有的五个开关（11111）即可紧急报警，系统将持续发出警报声直至按钮推下。

频率调节：通过开关（P4,P3，即 Tone）选择可将过低频率的声音调高（10），或将过高频率的声音调低（01）。

平滑声音：可通过开关（P5，即 Youhua）开启对声音的平滑处

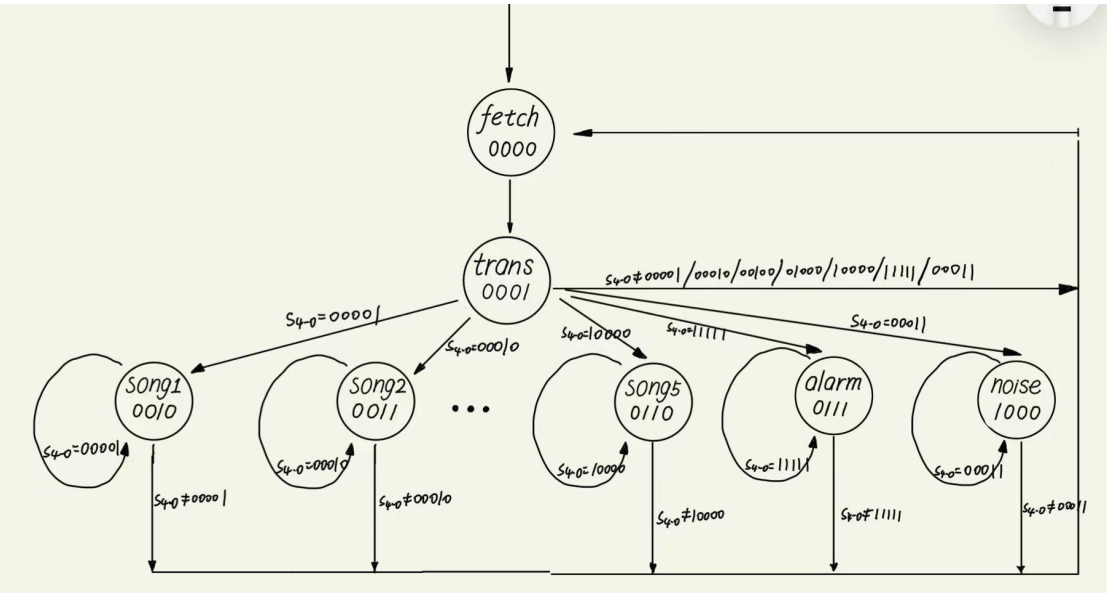
理，使声音更加连贯，听感提升。

低通滤波：可通过开关（P5）开启低通滤波功能，将高频声音自动过滤，保持原声输出。

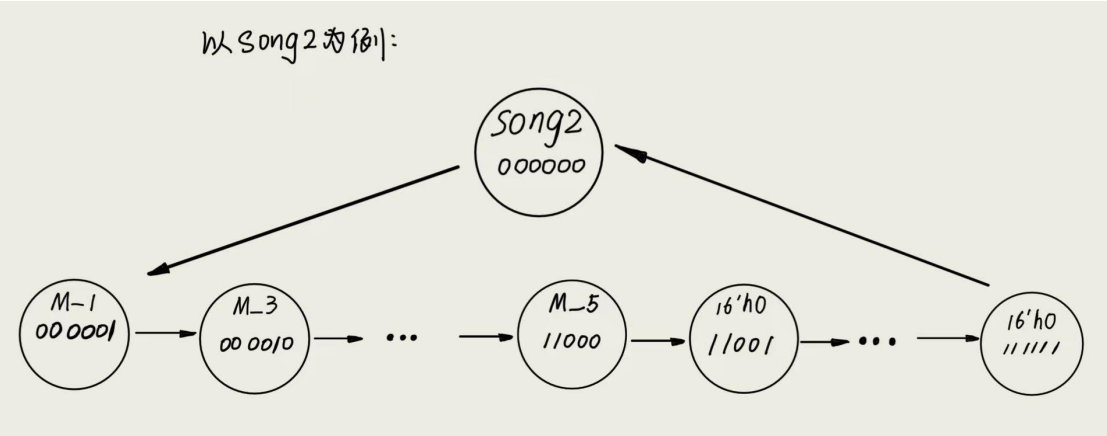
助听器主要需要实现的功能是对声音的调节，由于收声较为困难，我们选择在板子内提前谱写歌曲，并对歌曲进行处理。

二、 数字系统设计：

一级状态图：通过开关控制状态转换选择对应歌曲播放。

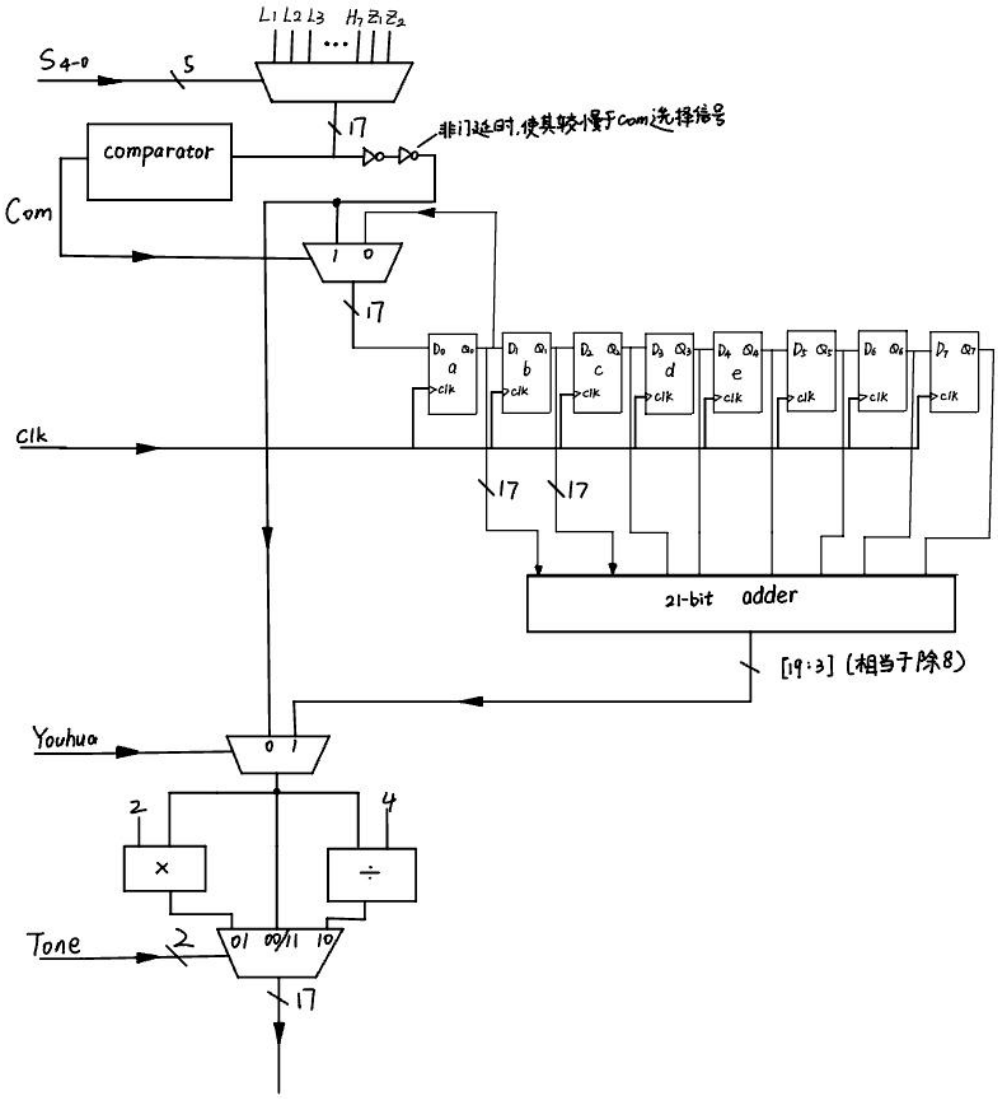


二级状态图：通过状态累加来实现音阶的循环顺序播放。



数据通路：

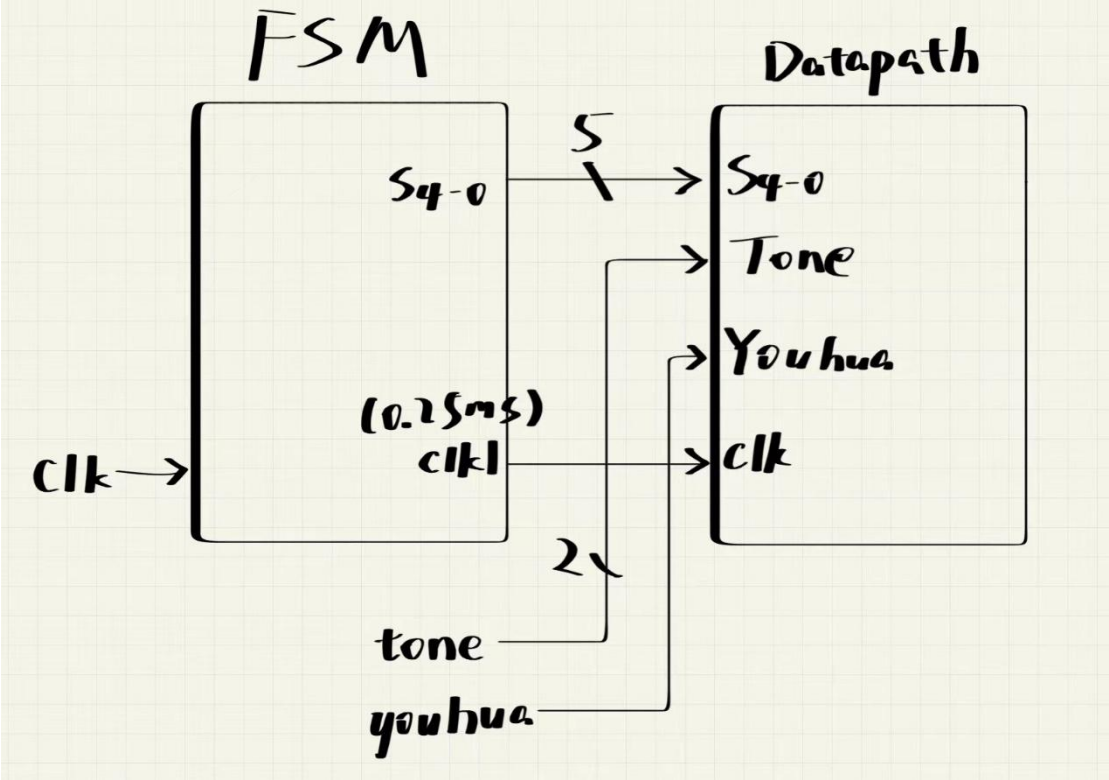
当音阶小于 17'b0100000010111100 时 comparator 输出 0



控制字：二级状态中每个状态对应一个音阶，通过控制字 S4-0 来选择对应音阶，举低音 do,re,me 为例，其他同理不再一一列举。

音阶	S4	S3	S2	S1	S0
L_1	0	0	0	0	1
L_2	0	0	0	1	0
L_3	0	0	0	1	1

其余控制字 Tone,Youhua 均为外部输入。因为我们是循环播放歌曲无 reset 需求。



三、 外设原理及实现：

温度传感器

我们选择使用 DS18B20 数字温度传感器，它通过单总线接口与主设备进行来回通信。

DS18B20 内置了一个温度传感器和一个模数转换器（ADC），可以将温度以数字信号的形式输出。

温度测量范围为 -55°C 到 +125°C，精度为 ±0.5°C（在 -10°C 到 +85°C 范围内），因为我们将温度传感器用于耳道测温，

对精度要求并不高，所以我们只保留了正温度部分的整数部分。

我们保留整数部分，将串行二进制码改为十进制 BCD 代码，再

温度寄存器格式 图 2

	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
LS Byte	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	bit 15	bit 14	bit 13	bit 12	bit 11	bit 10	bit 9	bit 8
MS Byte	S	S	S	S	S	2^6	2^5	2^4

TEMPERATURE (°C)	DIGITAL OUTPUT (BINARY)	DIGITAL OUTPUT (HEX)
+125	0000 0111 1101 0000	07D0h
+85*	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FFF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Fh
-55	1111 1100 1001 0000	FC90h

通过数码管表示数据。

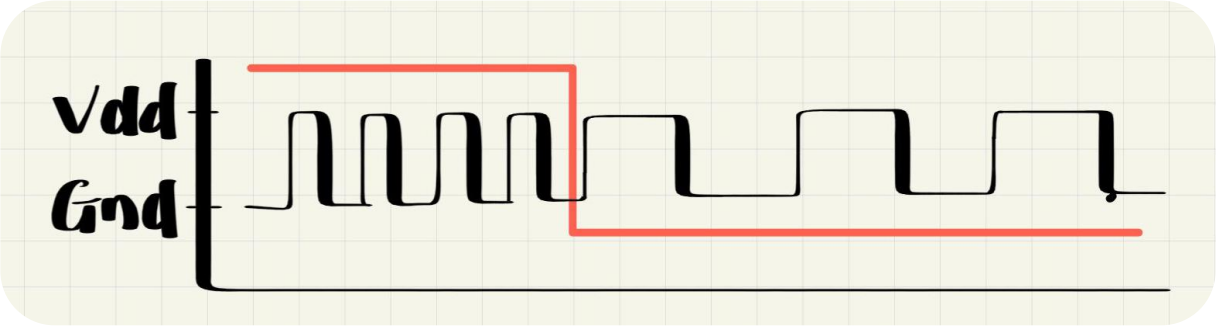
DS18B20 有外部电源模式和寄生电源模式，寄生电源模式下，传感器可以从数据线获取电源，我们使用寄生电源与板子连接。

外接音箱

使用板子自带的 PDM，即脉冲密度调制，使用二进制数 0,1 来表示模拟信号。在 PDM 信号中，模拟信号的幅值使用输出脉冲对应区域的密度表示。也就是说，1 的密度越大，代表该区域对应的模拟

信号幅值越大（如下图）。

因此，我们小组选择使用不同分频器对信号取反来实现 1 密度的



变化，从而输出不同的音阶，即通过不同时长的分频器对 beep 取反来设定占空比，占空时长则由音阶决定。（截取部分代码展示）

```
assign beep = beep_r;
always@(posedge clk) begin
    count <= count + 1'b1;
```

count3 即对应音阶宽度。

定义了低中高的“Do Re Mi Fa So La Xi”共 21 个音阶。

```
if(count == count3) begin
    count <= 17'h0;
    beep_r <= !beep_r;
end
end
```

```
parameter  //(G大调乐谱参数)
            L_1 = 17'b11111001001110000,  //低音1
            L_2 = 17'b11011100111100100,  //低音2
            L_3 = 17'b11000101101101100,  //低音3
            L_4 = 17'b10111010001011100,  //低音4
```

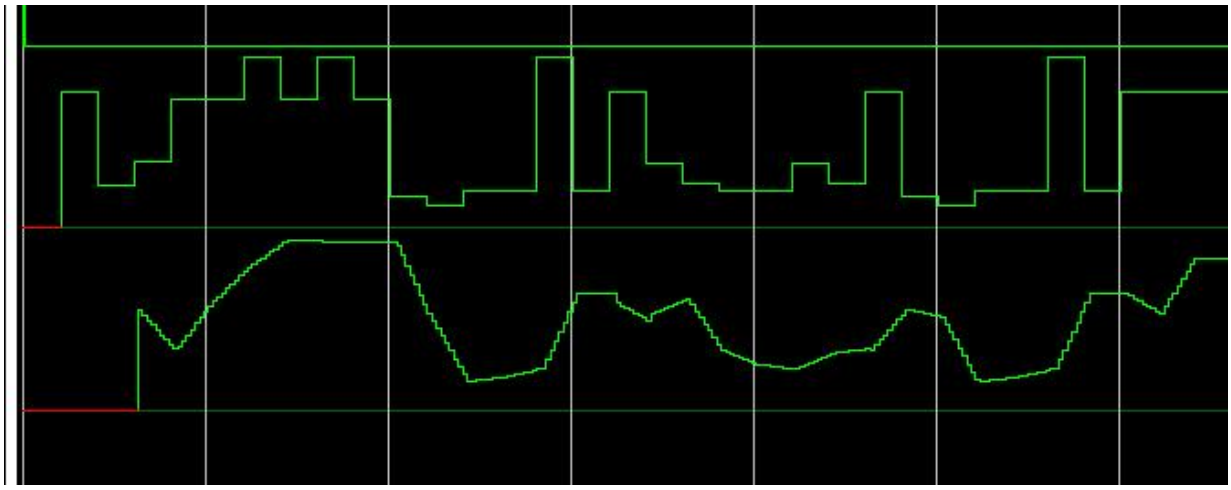
截取部分音阶展示

平滑处理

我们是通过不同频率的分频器来实现不同音阶的输出，所以对不同频率进行移动加权平均，我们可以实现声音的平滑处理。每过一个

时钟周期（设定为 0.25ms）我们就会把当前频率值串行输入到八个移位寄存器中，每个寄存器存储一个频率数值，将其求和后除以 8 即得到平滑后的声音。

如下仿真图所示，上曲线为平滑前声音曲线，下曲线为平滑后声音曲线。



四、 小组详细分工和占比：

乐曲谱写：吴溪语，孙钰淳。

测温功能：胡曹栋，吴溪语。

声音优化：孙钰淳，胡曹栋。

声音播放和处理：共同完成。

占比：吴溪语 1/3，孙钰淳 1/3，胡曹栋 1/3。

五、 总结：

在深入探索智能助听器设计的旅程中，我们深刻感受到了 fpga 开发的魅力和挑战，并从中汲取了宝贵的经验。设计过程中涉及的知

识和技能非常丰富，提升了我们使用 verilog 语言编程的能力，也锻炼了我们设计状态机、数据通路等的能力，让我们对微处理器有了更深入的认识。

首先，我们共同绘制了基本的状态转换过程，为总体设计奠定了思路基础。在设计智能助听器的过程中，状态机的设计尤为关键。状态机是控制逻辑的核心，它决定了系统在不同状态下的行为。我们通过精心设计分级状态机，确保了助听器在不同使用场景下都能表现出预期的功能。此外，数据通路的设计也是我们工作的重点之一。数据通路是执行算术和逻辑操作的硬件结构，我们通过优化数据通路的设计，提高了助听器处理音频信号的效率和准确性。

在实操过程中，我们遇到了各种各样的错误，其中部分问题在我们不懈的努力下成功解决了。刚开始接触温度传感器时，我们没有深入理解它的使用原理，对它的数据编码和传输过程存疑，得到了显示结果是混乱的。在仔细研读了硬件手册后，我们找到了引脚说明和数据编码的规则，对代码进行修改后，实现用数码管实时显示温度。另外，很多代码在思路上是正确的，例如声音播放，但是我们在接入音响后发现无法实现，在不断改错和仿真之后，我们发现代码中存在位宽错误等细节问题。我们在代码改错过程中积累了很多经验，很大程度上提高了实践能力和分析能力。

由于开题时我们没有正确考虑设计的难度，天马行空的提出了助听器的思想，这也造成了我们没有资料可查的尴尬处境。因此，面对种种难题，大部分时间我们都是一起讨论一起思考的。我们的分工很

明确，也每周固定三次时间进行讨论和交流，以确保了项目的顺利进行。过程中我们每个人也都发挥了自己的特长，互相补充，克服了一个又一个当时看起来不可能的任务，在一个良好的氛围中逐渐完成了大作业的设计。

在实现原定目标的过程中也存在着一些遗憾和不足，如原计划中的蓝牙模块由于时间和技术限制未能实现。我们使用的是开发板自带的蓝牙，在网上找到开源代码，并成功上电后，我们实现了手机和蓝牙的连接，数码管可以接收到手机传来的简单指令并显示。另外，我们也用 app-inventor 搭建好了基本的手机界面框架。然而，在做蓝牙模块将数据传输到手机界面的过程中，我们发现直接修改代码中的部分内容无法实现目标，在钻研了很久之后，也没有找到正确的解决方式，蓝牙模块遂夭折于此。蓝牙的失败让我们意识到自己对于这方面的理解还不够深入，在读懂硬件手册和灵活修改代码方面的技能还有待提高，这些需要我们在未来的学习和实践中格外注意并提升。另外，原定的声音处理是希望实现收声功能，但是由于材料购买出错，所以实时收声无法实现，改为自行编谱测试。对于声音平滑连接这一功能，从仿真图中看是实现了的，但是在实际播放音乐的过程中，平滑连接后的声音与原声差距较小，功能效果不够明显。

六、 完整代码：

分主机和从机完成。

主机：

```
`timescale 1ns / 1ps
module project1(
input    clk,
input      rst,
input[4:0]    switch,
input youhua,
input[1:0]    tone,
input wire [7:0] ooB,ttB,
output wire [7:0] oB,tB,
output  beep,                    //蜂鸣器输出
output    sd,
output wire    bcon,bcte);
        assign bcon=1'b1;
        assign bcte=1'b1;
        assign oB=ooB;
        assign tB=ttB;
reg beep_r;
reg[5:0] state;                  //乐谱状态机
reg[16:0]count,count1,count3;
wire[16:0] count4;
reg[25:0]count2;
assign sd=1'b1;
```

parameter //(G 大调乐谱参数)

```
L_1 = 17'b11111001001110000, //低音 1
L_2 = 17'b11011100111100100, //低音 2
L_3 = 17'b11000101101101100, //低音 3
L_4 = 17'b10111010001011100, //低音 4
L_5 = 17'b10100110100110000, //低音 5
L_6 = 17'b10010011110110110, //低音 6
L_7 = 17'b10000011011001111, //低音 7
M_1 = 17'b01111101011000000, //中音 1
M_2 = 17'b01101101000100010, //中音 2
M_3 = 17'b01100001111101111, //中音 3
M_4 = 17'b01011101000100010, //中音 4
M_5 = 17'b01010011011010001, //中音 5
M_6 = 17'b01001001010011000, //中音 6
M_7 = 17'b01000000101110111, //中音 7
H_1 = 17'b0111110101100000, //高音 1
H_2 = 17'b0110110100010010, //高音 2
H_3 = 17'b0110000111111000, //高音 3
H_4 = 17'b0101110100010010, //高音 4
H_5 = 17'b0101001101101000, //高音 5
H_6 = 17'b0100100101001100, //高音 6
H_7 = 17'b0100000010111100, //高音 7
```

```
                    Z_7 = 17'b0001000000101111, //噪音 7
                    Z_1 = 17'b0001111101011000; //噪音 1
parameter  TIME = 30000000;    //每种音阶持续时长为 300ms

assign beep = beep_r;
always@(posedge clk) begin
    count <= count + 1'b1;
    case(tone)
    2'b01:begin
    if(count == 2*count3) begin   //输出调低音
        count <= 17'h0;
        beep_r <= !beep_r;          //实际上每个周期分别包括等时长的
高电位和低电位，一高一低反复循环，形成不同频率的声音
    end
    end
    2'b10:begin
    if(count == count3/4) begin   //输出调高音
        count <= 17'h0;
        beep_r <= !beep_r;          //实际上每个周期分别包括等时长的
高电位和低电位，一高一低反复循环，形成不同频率的声音
    end
    end
```

```
    default:begin
    if(count == count3) begin      //输出原音
        count <= 17'h0;
        beep_r <= !beep_r;         //实际上每个周期分别包括等时长的
高电位和低电位，一高一低反复循环，形成不同频率的声音
    end
    end
    endcase
end

always @(posedge clk) begin
    if(count2 < TIME)
        count2 = count2 + 1'b1;
    else begin
        count2 = 26'd0;
        if(state == 8'd50)
            state = 8'd0;
        else
            state = state + 1'b1;
case(switch)
5'b00001:begin
    case(state)//小星星
```

```
8'D0:count1=M_1;
8'D1:count1=M_1;
8'D2:count1=M_5;
8'D3:count1=M_5;
8'D4:count1=M_6;
8'D5:count1=M_6;
8'D6:count1=M_5;
8'D7:count1=M_4;
8'D8:count1=M_4;
8'D9:count1=M_3;
8'D10:count1=M_3;
8'D11:count1=M_2;
8'D12:count1=M_2;
8'D13:count1=M_1;
8'D14:count1=M_1;
8'D15:count1=M_5;
8'D16:count1=M_5;
8'D17:count1=M_4;
8'D18:count1=M_4;
8'D19:count1=M_3;
8'D20:count1=M_3;
8'D21:count1=M_2;
```

```
8'D22:count1=M_5;
8'D23:count1=M_5;
8'D24:count1=M_4;
8'D25:count1=M_4;
8'D26:count1=M_3;
8'D27:count1=M_3;
8'D28:count1=M_2;
8'D29:count1=M_1;
8'D30:count1=M_1;
8'D31:count1=M_5;
8'D32:count1=M_5;
8'D33:count1=M_6;
8'D34:count1=M_6;
8'D35:count1=M_5;
8'D36:count1=M_4;
8'D37:count1=M_4;
8'D38:count1=M_3;
8'D39:count1=M_3;
8'D40:count1=M_2;
8'D41:count1=M_2;
8'D42:count1=M_1;
default: count1 = 16'h0;
```

```
        endcase
        end
5'b00010:begin
          case(state)//暖暖
        8'D0:count1=M_1;
        8'D1:count1=M_3;
        8'D2:count1=M_4;
        8'D3:count1=M_5;
        8'D4:count1=M_5;
        8'D5:count1=M_3;
        8'D6:count1=M_4;
        8'D7:count1=M_5;
        8'D8:count1=M_5;
        8'D9:count1=M_6;
        8'D10:count1=M_7;
        8'D11:count1=H_1;
        8'D12:count1=M_3;
        8'D13:count1=M_3;
        8'D14:count1=M_4;
        8'D15:count1=M_5;
        8'D16:count1=M_5;
        8'D17:count1=M_5;
```

```
8'D18:count1=M_5;
8'D19:count1=M_6;
8'D20:count1=M_6;
8'D21:count1=M_5;
8'D22:count1=M_4;
8'D23:count1=M_5;
8'D24:count1=H_1;
8'D25:count1=H_1;
8'D26:count1=H_1;
8'D27:count1=H_1;
8'D28:count1=M_6;
8'D29:count1=M_6;
8'D30:count1=M_6;
8'D31:count1=M_7;
8'D32:count1=H_1;
8'D33:count1=H_1;
8'D34:count1=M_7;
8'D35:count1=M_7;
8'D36:count1=M_5;
8'D37:count1=M_5;
8'D38:count1=M_3;
8'D39:count1=M_4;
```

```
      8'D40:count1=M_5;
      8'D41:count1=M_5;
      8'D42:count1=M_6;
      8'D43:count1=M_7;
      8'D44:count1=H_1;
      8'D45:count1=H_1;
      8'D46:count1=M_7;
      8'D47:count1=M_6;
      8'D48:count1=M_5;
      default:count1=16'h0;
        endcase
end
5'b00100:begin
        case(state)//ZJU
    8'D0:count1=M_3;
      8'D1:count1=M_3;
      8'D2:count1=M_3;
      8'D3:count1=M_3;
      8'D4:count1=M_3;
      8'D5:count1=M_5;
      8'D6:count1=M_5;
      8'D7:count1=M_5;
```

```
8'D8:count1=M_1;
8'D9:count1=M_1;
8'D10:count1=M_2;
8'D11:count1=M_2;
8'D12:count1=M_2;
8'D13:count1=M_4;
8'D14:count1=M_4;
8'D15:count1=M_3;
8'D16:count1=M_3;
8'D17:count1=M_3;
8'D18:count1=M_5;
8'D19:count1=M_5;
8'D20:count1=M_6;
8'D21:count1=M_6;
8'D22:count1=M_4;
8'D23:count1=M_4;
8'D24:count1=M_6;
8'D25:count1=M_6;
8'D26:count1=M_6;
8'D27:count1=M_6;
8'D28:count1=M_7;
8'D29:count1=M_7;
```

```
8'D30:count1=M_6;
8'D31:count1=M_6;
8'D32:count1=M_5;
8'D33:count1=M_5;
8'D34:count1=H_1;
8'D35:count1=H_1;
8'D36:count1=M_7;
8'D37:count1=M_7;
8'D38:count1=M_5;
8'D39:count1=M_5;
8'D40:count1=M_3;
8'D41:count1=M_3;
8'D42:count1=M_6;
8'D43:count1=M_6;
8'D44:count1=M_2;
8'D45:count1=M_2;
8'D46:count1=M_3;
8'D47:count1=M_3;
8'D48:count1=M_2;
8'D49:count1=M_2;
  default: count1 = 16'h0;
  endcase
```

```
end
5'b01000:begin
case(state)

    8'D0:count1=M_5;   //我和我的祖国
    8'D1:count1=M_6;
    8'D2:count1=M_5;
    8'D3:count1=M_4;
    8'D4:count1=M_3;
    8'D5:count1=M_2;
    8'D6:count1=M_1;
    8'D7:count1=M_1;
    8'D8:count1=M_1;
    8'D9:count1=L_5;
    8'D10:count1=L_5;
    8'D11:count1=L_5;
    8'D12:count1=M_1;
    8'D13:count1=M_3;
    8'D14:count1=H_1;
    8'D15:count1=M_7;
    8'D16:count1=M_6;
    8'D17:count1=M_3;
```

```
8'D18:count1=M_5;
8'D19:count1=M_5;
8'D20:count1=M_5;
8'D21:count1=M_5;
8'D22:count1=M_5;
8'D23:count1=M_5;
8'D24:count1=M_6;
8'D25:count1=M_7;
8'D26:count1=M_6;
8'D27:count1=M_5;
8'D28:count1=M_4;
8'D29:count1=M_3;
8'D30:count1=M_2;
8'D31:count1=M_2;
8'D32:count1=M_2;
8'D33:count1=L_6;
8'D34:count1=L_6;
8'D35:count1=L_6;
8'D36:count1=L_7;
8'D37:count1=L_6;
8'D38:count1=L_5;
8'D39:count1=M_5;
```

```
    8'D40:count1=M_1;
    8'D41:count1=M_2;
    8'D42:count1=M_3;
    8'D43:count1=M_3;
    8'D44:count1=M_3;
    8'D45:count1=M_3;
    8'D46:count1=M_3;
    8'D47:count1=M_3;
    default:count1=16'h0;
endcase
end
5'b10000:begin
case(state)
    8'D0:count1 = M_3;   //外国名曲
    8'D1:count1=M_3;
    8'D2:count1=M_3;
    8'D3:count1=M_3;
    8'D4:count1=M_2;
    8'D5:count1=M_1;
    8'D6:count1=M_1;
    8'D7:count1=L_7;
    8'D8:count1=L_6;
```

```
8'D9:count1=L_6;
8'D10:count1=M_1;
8'D11:count1=M_3;
8'D12:count1=M_6;
8'D13:count1=M_6;
8'D14:count1=M_6;
8'D15:count1=M_6;
8'D16:count1=M_5;
8'D17:count1=M_4;
8'D18:count1=M_4;
8'D19:count1=M_3;
8'D20:count1=M_2;
8'D21:count1=M_2;
8'D22:count1=M_3;
8'D23:count1=M_4;
8'D24:count1=M_3;
8'D25:count1=M_4;
8'D26:count1=M_3;
8'D27:count1=M_5;
8'D28:count1=M_4;
8'D29:count1=M_3;
8'D30:count1=M_3;
```

```
        8'D31:count1=M_2;
        8'D32:count1=M_1;
        8'D33:count1=M_1;
        8'D34:count1=L_7;
        8'D35:count1=L_6;
        8'D36:count1=L_7;
        8'D37:count1=L_7;
        8'D38:count1=L_7;
        8'D39:count1=L_7;
        8'D40:count1=M_1;
        8'D41:count1=L_7;
        8'D42:count1=L_6;
        8'D43:count1=L_6;
        8'D44:count1=L_6;
        8'D45:count1=L_6;
        default:count1=16'h0;
endcase
end
5'b11111:begin
case(state)//报警
    8'D0:count1=L_1;
    8'D1:count1=H_1;
```

```
8'D2:count1=L_1;
8'D3:count1=H_1;
8'D4:count1=L_1;
8'D5:count1=H_1;
8'D6:count1=L_1;
8'D7:count1=H_1;
8'D8:count1=L_1;
8'D9:count1=H_1;
8'D10:count1=L_1;
8'D11:count1=H_1;
8'D12:count1=L_1;
8'D13:count1=H_1;
8'D14:count1=L_1;
8'D15:count1=H_1;
8'D16:count1=L_1;
8'D17:count1=H_1;
8'D18:count1=L_1;
8'D19:count1=H_1;
8'D20:count1=L_1;
8'D21:count1=H_1;
8'D22:count1=L_1;
8'D23:count1=H_1;
```

```
8'D24:count1=L_1;
8'D25:count1=H_1;
8'D26:count1=L_1;
8'D27:count1=H_1;
8'D28:count1=L_1;
8'D29:count1=H_1;
8'D30:count1=L_1;
8'D31:count1=H_1;
8'D32:count1=L_1;
8'D33:count1=H_1;
8'D34:count1=L_1;
8'D35:count1=H_1;
8'D36:count1=L_1;
8'D37:count1=H_1;
8'D38:count1=L_1;
8'D39:count1=H_1;
8'D40:count1=L_1;
8'D41:count1=H_1;
8'D42:count1=L_1;
8'D43:count1=H_1;
8'D44:count1=L_1;
8'D45:count1=H_1;
```

```
    8'D46:count1=L_1;
    8'D47:count1=H_1;
    8'D48:count1=L_1;
    8'D49:count1=H_1;
    8'D50:count1=L_1;
        endcase
end
5'b00011:begin
case(state)
      8'D0:count1=M_1;   //小星星噪声版（验证低通滤波功能）
      8'D1:count1=M_1;
      8'D2:count1=M_5;
      8'D3:count1=M_5;
      8'D4:count1=Z_1;
      8'D5:count1=M_6;
      8'D6:count1=M_5;
      8'D7:count1=M_4;
      8'D8:count1=M_4;
      8'D9:count1=M_3;
      8'D10:count1=Z_7;
      8'D11:count1=M_2;
      8'D12:count1=M_2;
```

```
8'D13:count1=M_1;
8'D14:count1=M_1;
8'D15:count1=Z_1;
8'D16:count1=M_5;
8'D17:count1=M_4;
8'D18:count1=M_4;
8'D19:count1=M_3;
8'D20:count1=M_3;
8'D21:count1=M_2;
8'D22:count1=M_5;
8'D23:count1=M_5;
8'D24:count1=M_4;
8'D25:count1=M_4;
8'D26:count1=M_3;
8'D27:count1=Z_1;
8'D28:count1=Z_1;
8'D29:count1=M_1;
8'D30:count1=M_1;
8'D31:count1=M_5;
8'D32:count1=M_5;
8'D33:count1=M_6;
8'D34:count1=M_6;
```

```
    8'D35:count1=M_5;
    8'D36:count1=Z_7;
    8'D37:count1=Z_7;
    8'D38:count1=M_3;
    8'D39:count1=M_3;
    8'D40:count1=Z_1;
    8'D41:count1=M_2;
    8'D42:count1=M_1;
    default: count1 = 16'h0;
    endcase
    end
default:count1=16'h0;
    endcase
    end
    end

weighted_average_filter w1(
.clk(clk),                  // 输入时钟
.reset(rst),                // 复位信号
.audio_in(count1),   // 输入声音信号，假设 17 位输入
.audio_out(count4)     // 输出声音信号，假设 17 位输出
);
```

```
always @(*)begin
if(youhua) count3<=count4;
else count3<=count1;
end

endmodule

module weighted_average_filter (
    input wire clk,                  // 输入时钟
    input wire reset,                // 复位信号
    input wire [16:0] audio_in,   // 输入声音信号，假设 17 位输入
    output reg [16:0] audio_out     // 输出声音信号，假设 17 位输出
);

// 内部变量定义
reg [16:0] a,b,c,d,e,f,g,h;   // 缓冲区数组，存储输入信号
reg [20:0] sum;       // 加权和，扩展为 21 位以防止溢出
reg clk1=0;   // 分频后的时钟信号
reg [25:0] count=0;   // 用于分频计数器，根据实际情况调整位宽

always@(posedge clk)begin
if(count==250000)begin
```

```
clk1=~clk1;
count=0;
end
else
count=count+1'b1;
end

always @(posedge clk1) begin
if(audio_in<17'b0100000010111100) begin//加权平均
        a<=a;//过滤高频音阶，用先前音阶代替
        b<=a;
        c<=b;
        d<=c;
        e<=d;
        f<=e;
        g<=f;
        h<=g;
        sum<=a+b+c+d+e+f+g+h;
        audio_out=sum[19:3];
        end
else begin
        a<=audio_in;//加权平均
```

```
            b<=a;
            c<=b;
            d<=c;
            e<=d;
            f<=e;
            g<=f;
            h<=g;
            sum<=a+b+c+d+e+f+g+h;
            audio_out=sum[19:3];
            end
end

endmodule
```

从机:

```
`timescale    1ns/1ns

module    ds18b20
(
      input     wire                sys_clk        ,
      input     wire                sys_rst_n      ,
      inout     wire                dq             ,
      output wire [7:0] oB,tB,
```

```
        output wire    bcon,bcte);
        wire        [19:0]      data_out;
        wire [3:0] bcone,bcten;
        wire [3:0] one,ten;
        assign bcon=bcone;
        assign bcte=bcten;
wire                    sign        ;
ds18b20_ctrl        ds18b20_ctrl_inst
(
        .sys_clk        (sys_clk    ),
        .sys_rst_n      (sys_rst_n),
        .dq             (dq         ),
        .data_out       (data_out ),
        .sign           (sign       )
);
wire a,b,c;

bin_dec(.clk(sys_clk),.bin(data_out[11:4]),.rst_n(sys_rst_n),.one(one),.ten
(ten),.hun(a),.count(b),.shift_reg(c)
        );

Nixielight n1(.B(one),.SEG(oB),.bC(bcone));
```

```
Nixielight n2(.B(ten),.SEG(tB),.bC(bcten));


endmodule


module bin_dec(clk,bin,rst_n,one,ten,hun,count,shift_reg
        );
input    [7:0] bin;
input              clk,rst_n;
output [3:0] one,ten;
output [3:0] count;
output [1:0] hun;
output [17:0]shift_reg;
reg        [3:0] one,ten;
reg        [1:0] hun;
reg        [3:0] count;
reg        [17:0]shift_reg=18'b000000000000000000;
//  计数部分
always @ ( posedge clk or negedge rst_n )
begin
  if( !rst_n )
      count<=0;
  else if (count==9)
```

```
    count<=0;
  else
    count<=count+1;
end
// 二进制转换为十进制 /
always @ (posedge clk or negedge rst_n )
begin
   if (!rst_n)
          shift_reg=0;
   else if (count==0)
          shift_reg={10'b0000000000,bin};
   else if ( count<=8)                          //实现 8 次移位操作
    begin
         if(shift_reg[11:8]>=5)                 //判断个位是否>5，如果是则+3
              begin
                   if(shift_reg[15:12]>=5) //判断十位是否>5，如果是则+3
                        begin
    shift_reg[15:12]=shift_reg[15:12]+2'b11;
    shift_reg[11:8]=shift_reg[11:8]+2'b11;
shift_reg=shift_reg<<1;   //对个位和十位操作结束后，整体左移
  end
                   else
```

```
            begin
                        shift_reg[15:12]=shift_reg[15:12];
shift_reg[11:8]=shift_reg[11:8]+2'b11;
shift_reg=shift_reg<<1;
  end
                end
          else
                begin
                    if(shift_reg[15:12]>=5)
                        begin
     shift_reg[15:12]=shift_reg[15:12]+2'b11;
     shift_reg[11:8]=shift_reg[11:8];
shift_reg=shift_reg<<1;
  end
                    else
            begin
                        shift_reg[15:12]=shift_reg[15:12];
shift_reg[11:8]=shift_reg[11:8];
shift_reg=shift_reg<<1;
  end
                end
    end
```

```
    end
always @ ( posedge clk or negedge rst_n )
begin
  if ( !rst_n )
    begin
        one<=0;
        ten<=0;
        hun<=0;
    end
  else if (count==9)    //此时 8 次移位全部完成，将对应的值分别赋给个,十,百位
    begin
        one<=shift_reg[11:8];
ten<=shift_reg[15:12];
hun<=shift_reg[17:16];
    end
end
endmodule

module    ds18b20_ctrl
(
        input      wire              sys_clk          ,
```

```
    input   wire        sys_rst_n   ,
    inout   wire        dq          ,   //数据总线
    output  wire [19:0] data_out    ,   //输出温度
    output  reg         sign            //输出温度符号位
);

parameter   S_INIT          =   3'd1,
            S_WR_CMD        =   3'd2,
            S_WAIT          =   3'd3,
            S_INIT_AGAIN    =   3'd4,
            S_RD_CMD        =   3'd5,
            S_RD_TEMP       =   3'd6;

parameter   WR_44CC_CMD =   16'h44cc; //跳过 ROM 及温度转换命令，低位在前
parameter   WR_BECC_CMD =   16'hbecc; //跳过 ROM 及读取温度命令，低位在前

parameter   S_WAIT_MAX  =   750000  ; //750ms

reg         clk_1us     ;   //分频时钟，单位时钟 1us
reg [4:0]   cnt         ;   //分频计数器
```

```
reg [2:0]     state          ;     //状态机状态
reg [19:0]    cnt_1us        ;     //微秒计数器
reg [3:0]     bit_cnt        ;     //字节计数器
reg [15:0]    data_tmp       ;     //读取 ds18b20 的温度
reg [19:0]    data           ;     //判断完正负后的温度
reg           flag_pulse     ;     //初始化存在脉冲标志信号
reg           dq_out         ;     //输出总线数据，即 FPGA 给的总线数据值
reg           dq_en          ;     //输出总线数据使能信号

//温度转换，由于数码管位数有限，在这里保留小数点后三位
assign data_out = data;

//当使能信号为 1 是总线的值为 dq_out 的值，为 0 时为高阻态
assign    dq    =     (dq_en ==1 ) ? dq_out : 1'bz;

//cnt:分频计数器
always@(posedge sys_clk or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
                cnt <=    5'b0;
        else        if(cnt == 5'd24)
                cnt <=    5'b0;
```

```
        else
                cnt <=    cnt + 1'b1;

//clk_1us：产生单位时钟为 1us 的时钟
always@(posedge sys_clk or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
                clk_1us <=    1'b0;
        else        if(cnt == 5'd24)
                clk_1us <=    ~clk_1us;
        else
                clk_1us <=    clk_1us;

//cnt_1us：1us 时钟计数器，用于状态跳转
always@(posedge clk_1us or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
                cnt_1us <=    20'b0;
        else                if(((state==S_WR_CMD   ||   state==S_RD_CMD   ||
state==S_RD_TEMP)
            &&          cnt_1us==20'd64)          ||          ((state==S_INIT          ||
state==S_INIT_AGAIN) &&
            cnt_1us==20'd999)            ||            (state==S_WAIT            &&
cnt_1us==S_WAIT_MAX))
```

```
            cnt_1us <=    20'b0;
        else
            cnt_1us <=    cnt_1us +      1'b1;

//bit_cnt：bit 计数器，写 1bit 或读 1bit 加 1，一次写完之后清零
always@(posedge clk_1us or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            bit_cnt    <=    4'b0;
        else       if((state == S_RD_TEMP || state == S_WR_CMD ||
              state == S_RD_CMD) && (cnt_1us == 20'd64 && bit_cnt ==
4'd15))
            bit_cnt    <=    4'b0;
        else       if((state == S_WR_CMD || state == S_RD_CMD ||
                                        state == S_RD_TEMP) && cnt_1us
== 20'd64)
            bit_cnt    <=    bit_cnt + 1'b1;

//初始化存在脉冲标志信号：初始化状态时，当总线发来存在脉冲才
能初始化成功
always@(posedge clk_1us or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            flag_pulse    <=    1'b0;
```

```
        else        if(cnt_1us == 20'd570 && dq == 1'b0 && (state == S_INIT ||
                                                                    state        == S_INIT_AGAIN))
            flag_pulse   <=   1'b1;
        else      if(cnt_1us == 999)
            flag_pulse   <=   1'b0;
        else
            flag_pulse   <=   flag_pulse;

//状态跳转
always@(posedge clk_1us or   negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            state     <=   S_INIT;
        else
            case(state)
        //初始化最小时间为 960us
            S_INIT:   //收到存在脉冲且时间需大于 960us 跳转
                if(cnt_1us == 20'd999   && flag_pulse == 1'b1 )
                    state     <=   S_WR_CMD;
                else
                    state     <=   S_INIT;
```

```
S_WR_CMD:     //发送完跳过 ROM 和温度转换命令后跳转
    if(bit_cnt == 4'd15 && cnt_1us == 20'd64 )
        state     <=    S_WAIT;
    else
        state     <=    S_WR_CMD;
S_WAIT: //等待 750ms 后跳转
    if(cnt_1us == S_WAIT_MAX)
        state     <=    S_INIT_AGAIN;
    else
        state     <=    S_WAIT;
S_INIT_AGAIN:     //再次初始化后跳转
    if(cnt_1us == 20'd999    && flag_pulse == 1'b1 )
        state     <=    S_RD_CMD;
    else
        state     <=    S_INIT_AGAIN;
S_RD_CMD:     //发送完跳过 ROM 和读取温度命令后跳转
    if(bit_cnt == 4'd15 && cnt_1us == 20'd64)
        state     <=    S_RD_TEMP;
    else
        state     <=    S_RD_CMD;
S_RD_TEMP:    //读完 2 字节的温度后跳转
    if(bit_cnt == 4'd15 && cnt_1us == 20'd64)
```

```
                state     <=    S_INIT;
            else
                state     <=    S_RD_TEMP;
        default:
                state     <=    S_INIT;
        endcase

//给各状态下的总线相应的时序
always@(posedge clk_1us or    negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        begin
            dq_out    <=    1'b0;
            dq_en     <=    1'b0;
        end
    else
        case(state)
    //初始化是最小 480us 低电平，然后释放总线
        S_INIT:
            if(cnt_1us < 20'd499)
                begin
                    dq_out    <=    1'b0;
                    dq_en     <=    1'b1;
```

```
            end
        else
            begin
                dq_out   <=   1'b0;
                dq_en    <=   1'b0;
            end
//每一个写时段最少有 60us 的持续时间和最少 1us 的恢复时间
//写 0：总线拉低后一直拉低，最少 60us
//写 1：总线拉低后必须在 15us 内释放总线
    S_WR_CMD:
        if(cnt_1us > 20'd62)
            begin
                dq_out   <=   1'b0;
                dq_en    <=   1'b0;
            end
        else      if(cnt_1us <= 20'b1)
            begin
                dq_out   <=   1'b0;
                dq_en    <=   1'b1;
            end
        else      if(WR_44CC_CMD[bit_cnt] == 1'b0)
            begin
```

```
                dq_out   <=   1'b0;
                dq_en    <=   1'b1;
            end
        else      if(WR_44CC_CMD[bit_cnt] == 1'b1)
            begin
                dq_out   <=   1'b0;
                dq_en    <=   1'b0;
            end
//为适应寄生电源，温度转换命令后将总线拉高
    S_WAIT:
        begin
            dq_out   <=   1'b1;
            dq_en    <=   1'b1;
        end
//与第一次初始化时序一致
    S_INIT_AGAIN:
        if(cnt_1us < 20'd499)
            begin
                dq_out   <=   1'b0;
                dq_en    <=   1'b1;
            end
        else
```

```
                begin
                    dq_out    <=    1'b0;
                    dq_en     <=    1'b0;
                end
//与发送跳过 ROM 和读取温度命的时序一致
        S_RD_CMD:
            if(cnt_1us > 20'd62)
                begin
                    dq_out    <=    1'b0;
                    dq_en     <=    1'b0;
                end
            else        if(cnt_1us <= 20'b1)
                begin
                    dq_out    <=    1'b0;
                    dq_en     <=    1'b1;
                end
            else        if(WR_BECC_CMD[bit_cnt] == 1'b0)
                begin
                    dq_out    <=    1'b0;
                    dq_en     <=    1'b1;
                end
            else        if(WR_BECC_CMD[bit_cnt] == 1'b1)
```

```
                begin
                    dq_out   <=   1'b0;
                    dq_en    <=   1'b0;
                end
        //拉低总线超过 1us 后释放总线
            S_RD_TEMP:
                if(cnt_1us <=1)
                    begin
                        dq_out   <=   1'b0;
                        dq_en    <=   1'b1;
                    end
                else
                    begin
                        dq_out   <=   1'b0;
                        dq_en    <=   1'b0;
                    end
            default:;
            endcase

//data_tmp:读出温度，寄存在 data_tmp 里
always@(posedge clk_1us or   negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
```

```
            data_tmp        <=    12'b0;
            //总线拉低后数据有效时间为 15us
        else        if(state == S_RD_TEMP && cnt_1us == 20'd13)
            data_tmp      <=    {dq,data_tmp[15:1]};
        else
            data_tmp        <=    data_tmp;

//温度判断，输出温度
always@(posedge clk_1us or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            data        <=    20'b0;
        else        if(data_tmp[15] == 1'b0 && state == S_RD_TEMP &&
                                    cnt_1us  ==  20'd60  &&  bit_cnt  ==
4'd15)
            data        <=    data_tmp[10:0];
        else        if(data_tmp[15] == 1'b1 && state == S_RD_TEMP &&
                                    cnt_1us  ==  20'd60  &&  bit_cnt  ==
4'd15)
            data        <=    ~data_tmp[10:0] + 1'b1;

//温度判断，输出符号位
always@(posedge clk_1us or    negedge sys_rst_n)
```

```
        if(sys_rst_n == 1'b0)
            sign      <=   1'b0;
        else      if(data_tmp[15] == 1'b0 && state == S_RD_TEMP &&
                                    cnt_1us  ==  20'd60  &&  bit_cnt  ==
4'd15)
            sign      <=   1'b0;
        else      if(data_tmp[15] == 1'b1 && state == S_RD_TEMP &&
                                    cnt_1us  ==  20'd60  &&  bit_cnt  ==
4'd15)
            sign      <=   1'b1;

endmodule

module Nixielight(input[3:0] B,output reg [7:0]SEG,output [3:0]bC);
always@(*) begin
case (B)
   4'b0000: SEG=8'b11111100;
   4'b0001: SEG=8'b01100000;
   4'b0010: SEG=8'b11011010;
   4'b0011: SEG=8'b11110010;
   4'b0100: SEG=8'b01100110;
   4'b0101: SEG=8'b10110110;
```

```
    4'b0110: SEG=8'b10111110;
    4'b0111: SEG=8'b11100000;
    4'b1000: SEG=8'b11111110;
    4'b1001: SEG=8'b11110110;
endcase
end
  assign bC=15;
endmodule
```